



Abel Diaz Gonzalez

Department / Lab Name
Your University or Institution

June 2026

Outline

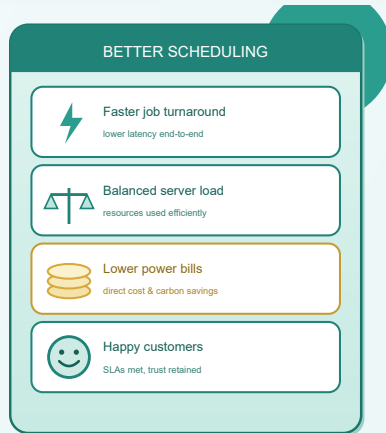
1 Background

2 MDP Formulation

Why Better Scheduling?



Smart
Scheduler



Data centers use 1–2% of global electricity
smarter scheduling directly cuts cost and carbon

Greedy Scheduling Baselines

Three greedy rules — three different orderings of the same jobs

each box = one job • width = run time • height = resource demand • colours track the same job across rows



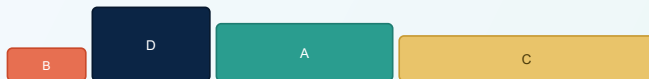
FCFS

first-come,
first-served
order: arrival



SJF

shortest
job first
order: run time



Smallest First

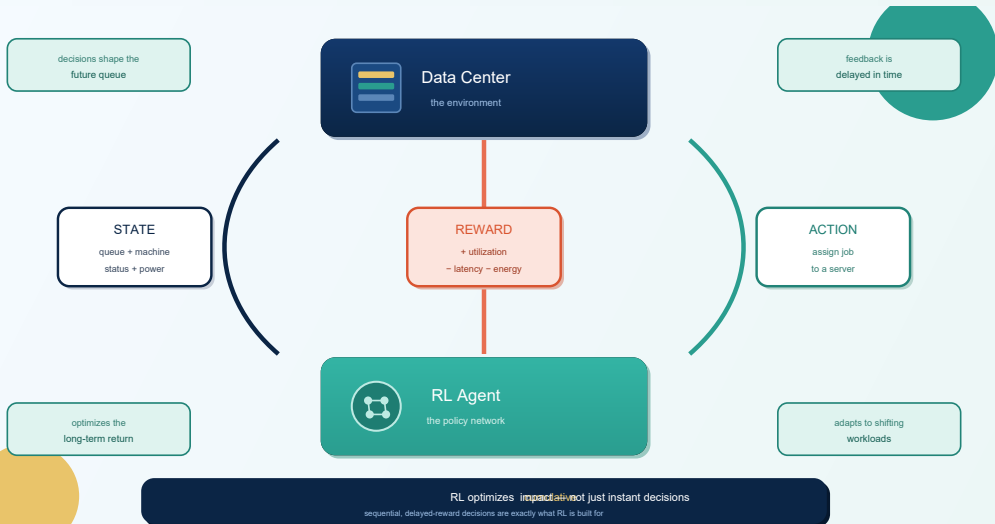
least resource
order: height



time / position in schedule

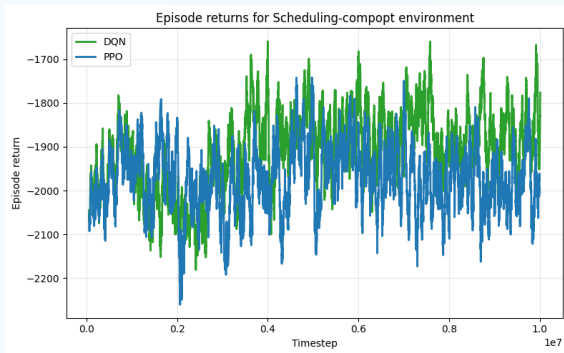
Simple and fast — ~~but only looks at the current queue~~ ignoring how today's choice shapes tomorrow

Why RL for Scheduling?



Does the RL Agent Actually Learn?

CompOpt environment



PPO & DQN returns stay flat —
the agents **fail to learn**

SchOpt environment (ours)



Returns climb steadily —
the agent **clearly learns**

Same algorithms (PPO & DQN), opposite outcome

So what did we actually change?

It wasn't the agent — it was the environment.

Defining the Problem as an MDP

$$\mathcal{M} = (\underbrace{\mathcal{S}}_{\text{state}}, \underbrace{\mathcal{A}}_{\text{action}}, \underbrace{P}_{\text{transition}}, \underbrace{R}_{\text{reward}}, \underbrace{\gamma}_{\text{discount}})$$

Component	Earlier (v1)	Now (v2)
State	12-dim aggregated	$2K+2$ per-job, normalised
Action	Discrete($K+1$) promote	unchanged
Transition	Δt step, stochastic arrivals	unchanged
Reward	$w_u u_t - w_Q Q_t$ (unbounded)	$-w \frac{\min(D_t, n_{\text{free}})}{N} \in [-1, 0]$
Horizon	truncate at T or drained	unchanged

Notation: N = total nodes | n_{free} = idle nodes | K = visible queued jobs | D_t = queue demand | u_t = utilisation |
 Q_t = queue length | T = horizon

Earlier (v1): aggregated summary

$s_t \in \mathbb{R}^{12}$: cluster stats + queue *means*.

- ▶ Cluster stats: $u_t, n_{\text{free}}, Q_t, \dots$
- ▶ Queue summarised by means $\bar{q}_t, \bar{w}_t$
- ▶ No per-job detail

Now (v2): per-job features

$s_t \in \mathbb{R}^{2K+2}$, one entry per visible job:

$$s_t = \left[\frac{t}{T}, \frac{n_{\text{free}}}{N}, \left(\frac{q_i}{N} \right)_{i=1}^K, \left(\frac{w_i}{T} \right)_{i=1}^K \right]$$

- ▶ Exposes *each* visible job, not just a mean
- ▶ Empty slots padded with -1 sentinel

Why: per-job observation lets the policy reason about *which* job to promote, not just queue averages.

Notation: u_t = utilisation | n_{free} = idle nodes | Q_t = queue length | $\bar{q}_t, \bar{w}_t$ = queue means |
 q_i, w_i = per-job size / wait | t, T = elapsed / horizon | N = total nodes | K = visible jobs

v1: util. — queue

$$r_t = w_u u_t - w_Q Q_t$$

- ▶ Rewards busy nodes, penalises backlog
- ▶ Penalty uses **raw** queue count Q_t
- ▶ **Unbounded**: scales with workload size

v2: schedulable-idle

$$r_t = -w \frac{\min(D_t, n_{\text{free}})}{N}$$

- ▶ Charges only for idle nodes the queue *could* use
- ▶ Bounded $[-1, 0]$, load-agnostic

v2: priority slowdown

$$r_t^p = -w_p \sum_j \widehat{\frac{p_j}{T_j}}$$

- ▶ Summed over **running & queued** jobs (p_j =priority, T_j =runtime)
- ▶ Bounded $[-1, 0]$; favours high-priority, short jobs

Why: targets head-of-line blocking directly with a well-scaled signal.

Notation: u_t = cluster utilisation | Q_t = raw queue length | D_t = total queue demand (nodes) | n_{free} = idle nodes | N = total nodes

Why These Changes Matter

From v1 to v2 we gain:

- ▶ **Bounded signals:** state in $[0, 1]$, reward in $[-1, 0]$
- ▶ **Load invariance:** no dependence on raw queue size or job distribution
- ▶ **Richer observation:** per-job features instead of a single mean
- ▶ **Direct objective:** penalise *schedulable* idle capacity (head-of-line blocking)

Net effect

A policy that learns to **promote jobs that fit** the free nodes, draining the queue faster, and that **transfers** across workloads without re-tuning reward weights.

Thank You

Background section only (v1)

A single **discrete** choice over the visible queue:

$$\mathcal{A} = \{0, 1, 2, \dots, K\} = \text{Discrete}(K+1)$$

- ▷ $a_t = 0$ (**no-op**): leave the queue ordering unchanged.
- ▷ $a_t = k \in \{1, \dots, K\}$: **promote** the k -th visible queued job to the front, so it is considered first on the next dispatch.

Key idea

The agent never dispatches jobs directly; it only *reprioritises*. The underlying scheduler then greedily packs jobs that fit onto free nodes.

Notation: K = number of visible queued jobs | a_t = action at step t | k = chosen queue position

Transition $P(s_{t+1} \mid s_t, a_t)$

(unchanged)

Each step advances the simulation by Δt seconds:

- ➊ **Apply the action:** if $a_t = k > 0$, move that job to the head of the queue.
- ➋ **Step the scheduler** by Δt : dispatch queued jobs onto free nodes (greedy packing), advance running jobs, release nodes from jobs that finish.
- ➌ **New arrivals:** jobs arrive stochastically (Poisson-like inter-arrival times set by the *offered load*).

Source of stochasticity

Job arrivals and the workload sampled each episode make the next state only *partially* predictable from (s_t, a_t) .

Notation: Δt = simulation step (seconds) | a_t = action at step t | k = promoted job index

An episode **truncates** when either

$t \geq T$ (24 h elapsed) or queue & running sets are empty (drained).

There is no absorbing failure state, so `terminated` is always `False`; episodes end only by truncation.

Learning objective

The agent learns a policy $\pi_\theta(a_t | s_t)$ maximising the expected discounted return:

$$\max_{\theta} \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right]$$

i.e. **keep the cluster busy** while draining the queue.

Notation: t, T = elapsed / horizon time | H = steps per episode | γ = discount factor | r_t = reward at step t